

Computer Networking Skills Assessment

HTB Academy Lab Environment · Chapters 1 – 3

Manish Burdak

HTB Academy · Network Fundamentals

Assessment Overview

- Chapter 1 – Keep me in the Loop: Explore network interfaces (ifconfig, netstat)
- Chapter 2 – Having Tuns of Fun: VPN tunnel (tun0), ping, nmap port scanning
- Chapter 3 – Target Acquired (Optional): FTP & HTTP protocol deep-dive
- Tools used: ifconfig, netstat, ip route, ping, nmap, netcat (nc)
- Platform: HTB Academy Pwnbox running Parrot OS (accessed via web browser)

Keep Me in the Loop

Network Interfaces & the Loopback Address

Viewing Network Interfaces — `ifconfig -a`

- Command: `ifconfig -a` → shows ALL interfaces (including inactive ones)
- Three interfaces on Pwnbox: `ens3`, `lo`, `tun0`
- `ens3`: physical Ethernet — public IP (209.50.61.235), MAC address, TX/RX stats
- `lo`: loopback — always 127.0.0.1, MTU 65536 (vs 1500 for real NICs)
- `tun0`: VPN tunnel — 10.x.x.x range, POINTOPOINT flag, no MAC address
- Flags: UP=active, BROADCAST=supports broadcast, RUNNING=ready, MULTICAST=supports multicast

The Loopback Interface (lo)

- IPv4: 127.0.0.1 | IPv6: ::1 | netmask: 255.0.0.0 (/8)
- MTU 65536 — much larger than Ethernet's 1500; no physical medium
- scopeid 'host' (not 'link') — only accessible by the local machine
- Layer-2 type shows 'Local Loopback' — no real hardware or MAC
- RX == TX packet counts: all traffic is self-sent and self-received
- Use-cases: local app testing, hidden internal services (e.g. databases), inter-process comms
- Example: e-commerce DB only accessible via 127.0.0.1; web app acts as API between users and DB

Checking Open Ports — netstat

netstat -tulnp4 (numeric)

- -t TCP connections
- -u UDP connections
- -l listening sockets only
- -n numeric IPs & ports
- -p show program name
- 4 IPv4 only
- 0.0.0.0 = all interfaces
- 127.0.0.1:5901 → Xtigervnc (VNC)
- 127.0.0.1:631 → CUPS printing
- 209.x.x.x:80 → public HTTP

netstat -tulp4 (hostnames)

- Remove -n to see hostnames
- 127.0.0.1 resolves to localhost
- ens3 IP resolves to Pwnbox hostname
- localhost:5901 → VNC service
- VNC is internal; port-forwarding exposes it via HTTP to browser
- Port forwarding: redirect traffic from one TCP/UDP port to another
- SSH port 22 open on 0.0.0.0 (all interfaces)

Port Forwarding

- Definition: redirect traffic arriving at one TCP/UDP port to a different port (or machine)
- Example: Windows host port 8888 → Linux VM's SSH port 22 (VM uses NAT, has no direct IP)
- Pwnbox uses port forwarding: HTTP (port 80) → VNC service (127.0.0.1:5901)
- This is how the browser-based desktop works — HTTP delivers the VNC session
- Common use-case: expose an internal service without giving it a public IP
- Topic extends into NAT, tunneling, and reverse proxies

Having Tuns of Fun

VPN Tunnels, Ping & Nmap

The tun0 Interface — VPN Tunnel

- tun0 = TUN (network TUNnel) virtual interface, used by VPNs
- IP: 10.10.14.x — private range, connects Pwnbox to HTB lab network
- Flag POINTOPOINT: no broadcast, direct point-to-point link
- No MAC address (unspec 00-00-00-00-...) — Layer-3 only, no Layer-2 hardware
- Verify route: `ip route get <target IP>` → output shows 'dev tun0'
- HTB Academy VPN downloadable at: academy.hackthebox.com/vpn (for own machine)
- Software: OpenVPN reads a .ovpn config file and creates the tun0 interface
- Allows you to reach lab machines as if you are on the same LAN

Testing Reachability — ping

- Protocol: ICMP (Internet Control Message Protocol) — Layer 3, no TCP/UDP ports
- Command: `ping -c 4 <target IP>` (-c = count; Linux pings forever without it)
- Output per line: `icmp_seq`, `ttl`, time (latency in ms)
- TTL (Time-To-Live): max hops a packet can take; decrements at each router
- Typical starting TTL: 64 (Linux/Mac), 128 (Windows), 255 (network devices)
- A TTL of 127 in response → target started at 128 → ~1 hop away
- time: round-trip latency; ~71 ms in example (acceptable over VPN)
- 0% packet loss = target is up and reachable

Port Scanning — nmap

- `nmap <target IP>` → default TCP SYN scan of top 1000 ports
- Open ports on Windows target: 21 (FTP), 80 (HTTP), 135/139/445 (SMB/RPC), 3389 (RDP), 5357 (WSD)
- 135, 139, 445: always open on Windows — NetBIOS + SMB
- 3389: Remote Desktop Protocol (RDP) — GUI remote access
- 5357: Web Services for Devices API — device discovery
- `nmap -p21,80 -sC -sV <IP>` → service version + default scripts on specific ports
- `-sV`: fingerprint service version | `-sC`: run default NSE scripts
- Port 21 reveals: Microsoft ftpd, Anonymous FTP login ALLOWED
- IMPORTANT: Never run nmap against hosts you don't own or have permission to scan

Target Open Ports — Quick Reference

Port	Protocol	Service	Notes
21	TCP	FTP	Anonymous login allowed — Microsoft ftpd
80	TCP	HTTP	Microsoft IIS 10.0 — User-Agent filtering active
135	TCP	MSRPC	Microsoft RPC Endpoint Mapper (Windows default)
139	TCP	NetBIOS-SSN	NetBIOS Session Service (Windows default)
445	TCP	Microsoft-DS	SMB (Server Message Block) — Windows file sharing
3389	TCP	RDP	Remote Desktop Protocol — GUI remote access
5357	TCP	WSDAPI	Web Services for Devices — Windows device discovery

Target Acquired

FTP & HTTP Protocols via Netcat

FTP Protocol Deep-Dive

Control Channel (port 21)

- `nc <target> 21` → banner: '220 Microsoft FTP Service'
- `USER anonymous` (sends username)
- `PASS anything` (anonymous accepts any password)
- `PASV` (switch to passive mode)
- `LIST` (list files in current dir)
- `RETR <file>` (retrieve/download file)
- Commands need `\r\n` → use Ctrl+V then Enter in netcat

Data Channel (dynamic port)

- FTP uses TWO channels: control (21) + data (dynamic)
- PASV response: (p1, p2, p3, p4, port_hi, port_lo)
- Calculate data port: $\text{port_hi} \times 256 + \text{port_lo}$
- Example: (194, 40) → $194 \times 256 + 40 = 49704$
- Open new terminal: `nc -v <target> <data_port>`
- Then send LIST or RETR on control channel
- Data flows on the data channel connection

FTP Passive Mode Port Calculation

- PASV response format: 227 Entering Passive Mode (h1,h2,h3,h4,p1,p2)
- h1–h4 = server IP octets | p1 & p2 = port encoding
- Data port formula: $p1 \times 256 + p2$
- Example 1: PASV → (10,129,233,197,194,40) → $194 \times 256 + 40 = 49704$
- Example 2: PASV → (10,129,233,197,194,50) → $194 \times 256 + 50 = 49714$
- Why split? FTP is old (1971) — designed when byte-saving was critical
- Workflow: PASV → calculate port → open netcat to data port → send LIST/RETR on control
- Each new transfer needs a fresh PASV + new data port connection

HTTP Protocol via Netcat

- Connect: `nc -v <target> 80` → port 80 open
- HTTP request format:
- `GET / HTTP/1.1` ← method + path + version
- `Host: <target IP>` ← required header (virtual hosting)
- `User-Agent: Server Administrator` ← custom header (bypass filter)
- (blank line) ← signals end of headers
- Server response includes: `HTTP/1.1 200 OK, Content-Type, Server: Microsoft-IIS/10.0`
- HTML is returned as plain text — browser would render it visually
- Flags/secrets can be hidden in HTML comments: `<!-- HTB{...} -->`
- Without correct User-Agent → request rejected (IIS configuration filter)

Key HTTP Headers

Header	Direction	Purpose	Example
Host	Request	Identifies the target virtual host	Host: 10.129.233.197
User-Agent	Request	Identifies the client making the request	User-Agent: Server Administrator
Content-Type	Response	MIME type of the response body	Content-Type: text/html
Accept-Ranges	Response	Supports partial content (byte ranges)	Accept-Ranges: bytes
Server	Response	Web server software info	Server: Microsoft-IIS/10.0
Content-Length	Response	Size of response body in bytes	Content-Length: 746
ETag	Response	Entity tag for caching validation	ETag: 5acd7854a179db1:0

Data Flow: Browser to Website

End-to-End Journey of an HTTP Request

Data Flow — Steps 1 to 4

- 1. WLAN Connection: laptop identifies SSID, authenticates (WPA2/WPA3), gets IP via DHCP
- 2. DHCP: router assigns private IP (e.g. 192.168.1.10), subnet mask, gateway, DNS server
- 3. DNS Resolution: laptop queries DNS server → www.example.com → 93.184.216.34
- 4. Encapsulation (OSI layers top-down):
 - App Layer: browser creates HTTP/HTTPS request
 - Transport Layer: wrapped in TCP segment (src/dst ports; HTTP=80, HTTPS=443)
 - Internet Layer: placed in IP packet (src: 192.168.1.10, dst: 93.184.216.34)
 - Link Layer: IP packet into Ethernet/Wi-Fi frame with MAC addresses
- ARP: laptop looks up router's MAC before sending the first frame

Data Flow — Steps 5 to 7

- 5. NAT at Router: private IP (192.168.1.10) replaced with public IP (203.0.113.45) in packet header
- Packet hops through multiple routers across the internet to reach 93.184.216.34
- 6. Server Receives Request:
 - Firewall checks: port 80/443 allowed?
 - Web server (Apache/Nginx/IIS) processes request, prepares HTML/CSS/images
 - Response travels back: src=93.184.216.34, dst=203.0.113.45
 - Home router NAT maps public IP back to 192.168.1.10
- 7. Decapsulation (OSI layers bottom-up):
 - Ethernet frame → IP packet → TCP segment → HTTP response → HTML rendered by browser

Assessment Question Answers — Quick Reference

- Q1: What IPv4 is used when a host sends traffic to itself? → 127.0.0.1
- Q2: Program listening on localhost:5901? → Xtigervnc
- Q3: Interface to interact with HTB lab targets? → tun0
- Q4: Tool to configure interfaces & show status? → ifconfig
- Q5: Tool to display network connections & routing? → netstat
- Optional Ex1: FTP command to retrieve a file? → RETR
- Optional Ex2: Bypass HTTP filter with header: User-Agent: Server Administrator
- Connect via nc, then: GET / HTTP/1.1 | Host: <IP> | User-Agent: Server Administrator | (blank line)

Command Cheat Sheet

Command	Purpose
<code>ifconfig -a</code>	List all network interfaces (including inactive)
<code>netstat -tulnp4</code>	Show listening TCP/UDP IPv4 ports with numeric output
<code>netstat -tulp4</code>	Same but with hostname/service name resolution
<code>ip route get <IP></code>	Show which interface/route a packet to <IP> would take
<code>ping -c 4 <IP></code>	Send 4 ICMP echo requests to test reachability
<code>nmap <IP></code>	Default TCP SYN scan of top 1000 ports
<code>nmap -p21,80 -sC -sV <IP></code>	Version + script scan of specific ports
<code>nc <IP> <port></code>	Raw TCP connection to a host:port
<code>nc -v <IP> <port></code>	Verbose TCP connection (shows open/closed status)